

Rapport de Travaux Pratiques

Big Data

MongoDB — Redis — ElasticSearch

Présenté par : Ikram KOUKOU

Encadrant : Lionel Antraigue

Formation : ISIMA F3

Année Universitaire : 2025-2026

26 novembre 2025

1 TP 1 : MongoDB

1.1 Introduction

Ce premier TP a pour objectif la découverte et la manipulation de MongoDB, une base de données NoSQL orientée documents. L'objectif principal était de créer une application web CRUD permettant de gérer un catalogue de manuscrits.

1.2 Mise en place de l'environnement

1.2.1 Configuration Docker

Le projet utilise Docker pour orchestrer les différents services nécessaires :

- **MongoDB** : Base de données NoSQL
- **Mongo Express/GUI** : Interface web de gestion
- **PHP 8.3** : Serveur d'application

La commande suivante a permis de lancer l'ensemble des containers :

```
1 docker compose up -d
```

Listing 1 – Lancement de l'environnement Docker

1.2.2 Configuration de PhpStorm

L'IDE PhpStorm a été configuré avec :

- Connexion à la base MongoDB (user : zz3f3, password : easyma)
- Installation du driver MongoDB
- Activation de la console de requêtes

1.3 Importation des données OpenData

Les manuscrits de la bibliothèque de Clermont-Ferrand ont été importés via la commande `mongoimport` :

```
1 ./mongoimport --collection=tp \  
2 mongodb://zz3f3:easyma@localhost:27017/tp \  
3 --authenticationDatabase=admin \  
4 --file=catalogue-manuscrits-bibliotheque-patrimoine.json \  
5 --jsonArray
```

Listing 2 – Import des données dans MongoDB

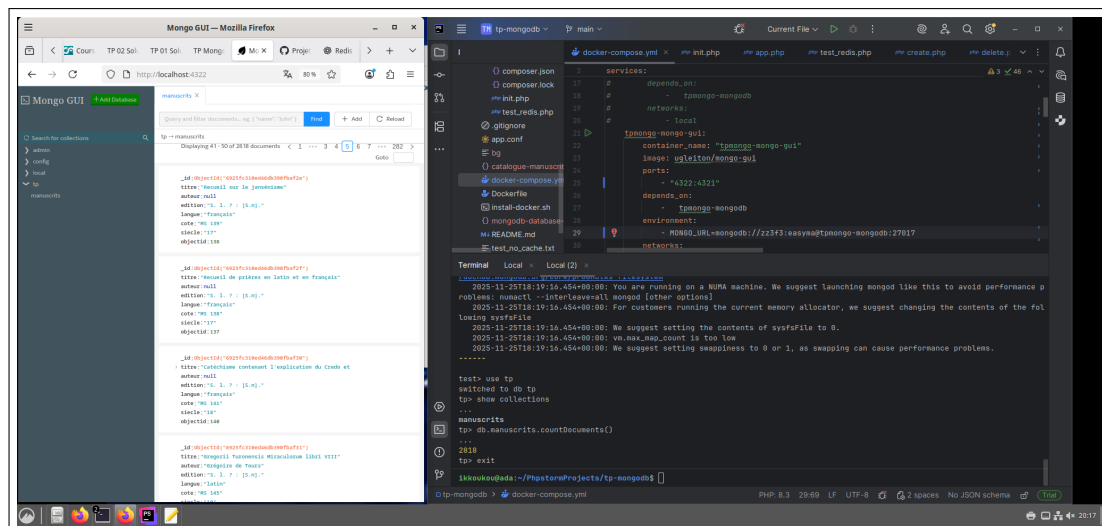


FIGURE 1 – Interface Mongo GUI - Visualisation des manuscrits importés (2818 documents)

1.4 Développement de l'application CRUD

1.4.1 Architecture de l'application

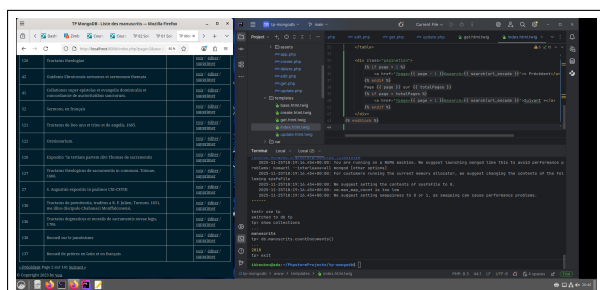
L'application web développée en PHP utilise :

- **Twig** : Moteur de templates
- **DotEnv** : Gestion de la configuration
- **mongodb/mongodb** : Librairie de connexion MongoDB

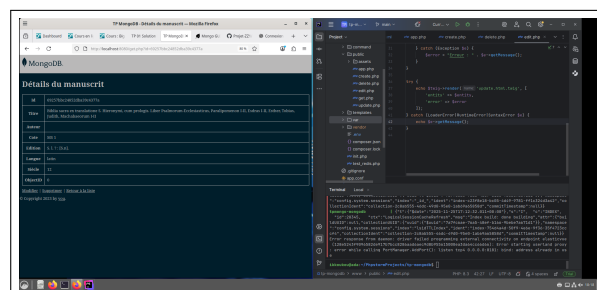
1.4.2 Implémentation du CRUD

Les quatre opérations CRUD ont été implémentées :

1. **Create** : Ajout d'un manuscrit via `create.php`
2. **Read** : Liste et visualisation détaillée via `app.php` et `get.php`
3. **Update** : Modification d'un manuscrit via `edit.php`
4. **Delete** : Suppression d'une entrée

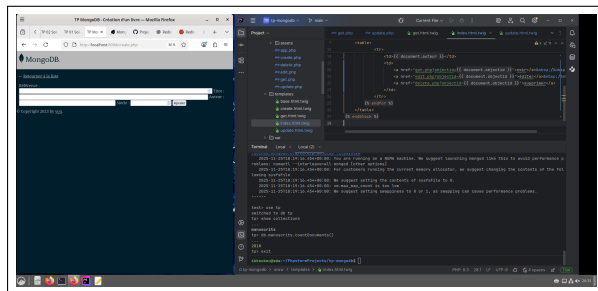


(a) Liste paginée des manuscrits avec recherche

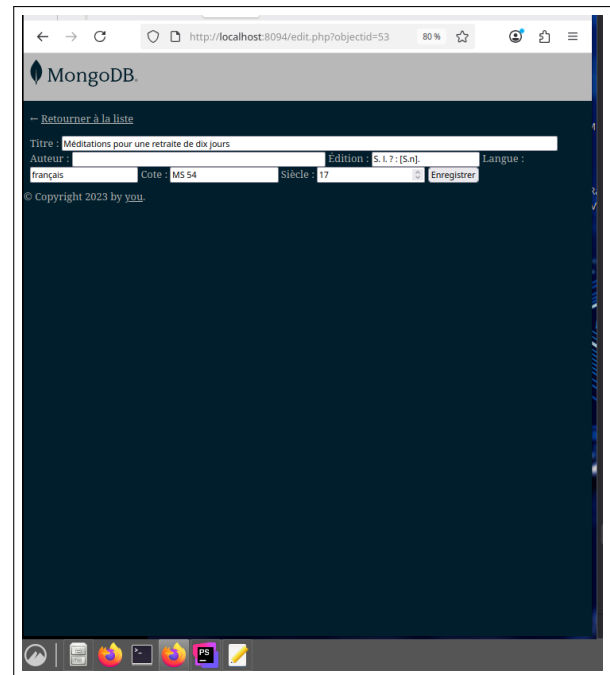


(b) Détails complets d'un manuscrit

FIGURE 2 – Interface de l'application MongoDB CRUD



(a) Formulaire de création d'un manuscrit



(b) Formulaire de modification avec données pré-remplies

FIGURE 3 – Fonctionnalités Create et Update du CRUD

Liste des manuscrits de la bibliothèque du Patrimoine de Clermont Auvergne Métropole

missale

Référence	Titre	Auteur	Actions
61	Missale Romanum		voir / éditer / supprimer
63	Missale, ad usum ecclesiae Claromontensis		voir / éditer / supprimer
65	Missale, ad usum ecclesiae Claromontensis		voir / éditer / supprimer
58	Missale Romanum		voir / éditer / supprimer
62	Missale, ad usum ecclesiae Claromontensis		voir / éditer / supprimer
71	Missale, ad usum ecclesiae Claromontensis		voir / éditer / supprimer
59	Missale, ad usum conventus Sancti Illidii		voir / éditer / supprimer
60	Missale, ad usum ecclesiae Claromontensis		voir / éditer / supprimer
72	Missale, ad usum ecclesiae Claromontensis ac Sancti Flori		voir / éditer / supprimer
64	Missale, ad usum ecclesiae Claromontensis		voir / éditer / supprimer
74	Missale, ad usum ecclesiae Claromontensis		voir / éditer / supprimer

Page 1 sur 1
© Copyright 2023 by you.

FIGURE 4 – Résultat de recherche avec le terme "missale"

1.5 Conclusion TP1

Ce premier TP a permis de comprendre les concepts fondamentaux de MongoDB :

- Structure de documents JSON/BSON
- Requêtes flexibles sans schéma fixe
- Manipulation via PHP et interface graphique
- Avantages pour les données semi-structurées

2 TP 2 : Redis

2.1 Introduction

Ce deuxième TP introduit Redis, une base de données NoSQL en mémoire utilisée principalement comme système de cache. L'objectif était d'améliorer les performances de l'application MongoDB développée précédemment.

2.2 Découverte de Redis

2.2.1 Installation et configuration

Deux nouveaux containers Docker ont été ajoutés :

- **redis :7.0.5** : Serveur Redis
- **redis-commander** : Interface web de visualisation

2.3 Intégration de Redis comme cache applicatif

2.3.1 Installation de la librairie Predis

```
1 composer require predis/predis
```

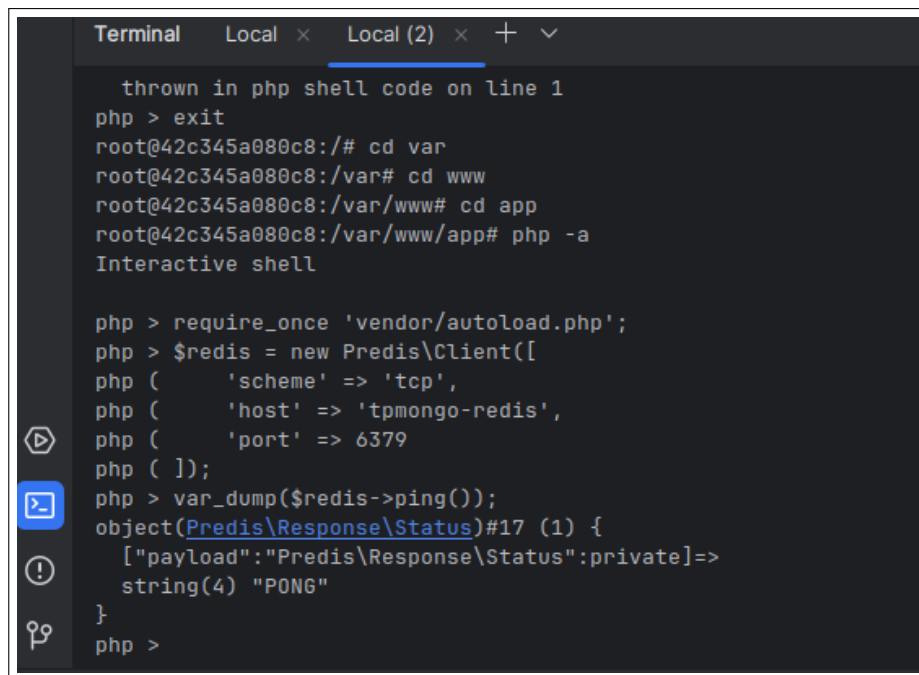
Listing 3 – Installation de Predis

2.3.2 Configuration

Le fichier `.env` a été complété avec :

```
1 REDIS_HOST=redis
2 REDIS_PORT=6379
3 REDIS_ENABLE=true
```

Listing 4 – Configuration Redis dans `.env`



```

Terminal  Local x  Local (2) x  +  v

thrown in php shell code on line 1
php > exit
root@42c345a080c8:/# cd var
root@42c345a080c8:/var# cd www
root@42c345a080c8:/var/www# cd app
root@42c345a080c8:/var/www/app# php -a
Interactive shell

php > require_once 'vendor/autoload.php';
php > $redis = new Predis\Client([
php (   'scheme' => 'tcp',
php (   'host' => 'tpmongo-redis',
php (   'port' => 6379
php (   ]);
php > var_dump($redis->ping());
object(Predis\Response\Status)#17 (1) {
  ["payload":"Predis\Response\Status":private]=>
    string(4) "PONG"
}
php >

```

FIGURE 5 – Test de connexion Redis via shell PHP interactif

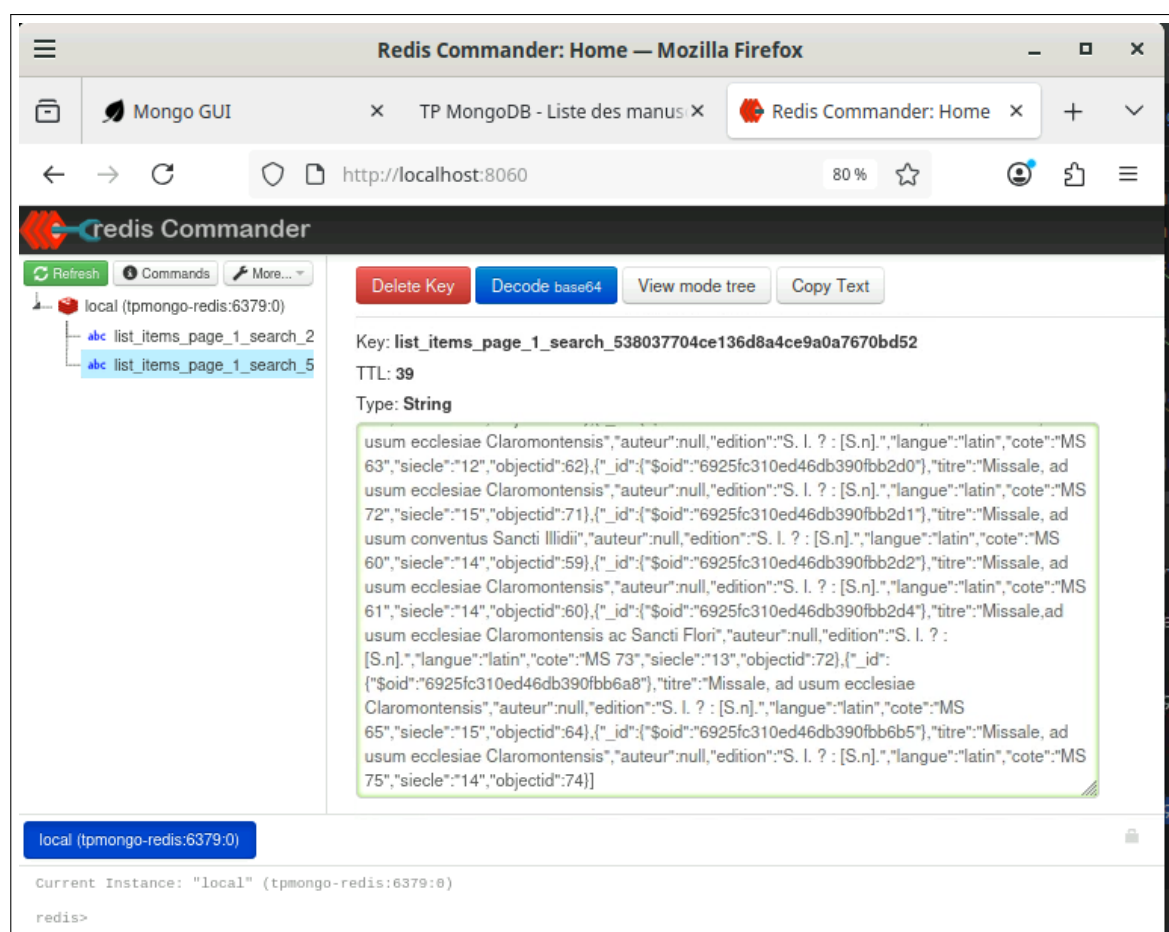


FIGURE 6 – Interface Redis Commander - Visualisation des données en cache

Fonction d'initialisation dans `init.php` :

```
1 function getRedisClient(): Predis\Client {  
2     return new Predis\Client([  
3         'scheme' => 'tcp',  
4         'host'   => $_ENV['REDIS_HOST'],  
5         'port'   => $_ENV['REDIS_PORT'],  
6     ]);  
7 }
```

Listing 5 – Initialisation du client Redis

2.4 Stratégie de mise en cache

2.4.1 Éléments mis en cache

- Liste complète des manuscrits : Clé manuscripts:all
- Détails individuels : Clé manuscript:id:{id}
- Résultats de recherche : Clé search:{query}
- TTL : 60 secondes

2.4.2 Implémentation

Le code suivant montre l'intégration complète du cache Redis dans `app.php` :

```
1 $redis = getRedisClient();  
2 $manager = getMongoDbManager();  
3  
4 // Pagination  
5 $perPage = 20;  
6 $page = isset($_GET['page']) ? max(1, (int)$_GET['page']) : 1;  
7 $skip = ($page - 1) * $perPage;  
8  
9 // Recherche  
10 $search = $_GET['search'] ?? '';  
11  
12 // Cle de cache unique pour cette page + recherche  
13 $cacheMode = 'mongo';  
14 $cacheKey = "list_items-{$cacheMode}_page-{$page}_search-" . md5(  
15     $search);  
16  
17 $cached = $redis ? $redis->get($cacheKey) : null;  
18  
19 if ($cached !== null) {  
20     // Recuperation depuis Redis (CACHE HIT)  
21     $list = json_decode($cached, true);  
22 } else {  
23     // Recuperation depuis MongoDB (CACHE MISS)  
24     $collection = $manager->selectCollection('manuscripts');  
25  
26     $cursor = $collection->find($filter, [  
27         'skip' => $skip,  
28         'limit' => $perPage
```



```

28     });
29
30     $list = [];
31     foreach ($cursor as $document) {
32         $list[] = $document->getArrayCopy();
33     }
34
35     // Sauvegarde dans Redis avec TTL de 60 secondes
36     if ($redis) {
37         $redis->setex($cacheKey, 60, json_encode($list));
38     }
39 }
40
41 $totalDocuments = $collection->countDocuments($filter);
42 $totalPages = ceil($totalDocuments / $perPage);

```

Listing 6 – Stratégie de cache dans app.php (extrait)

Cette implémentation utilise une clé de cache composite incluant le numéro de page et le hash MD5 de la recherche, permettant de mettre en cache chaque combinaison unique de requête.

2.5 Tests de performance avec K6

2.5.1 Configuration de l'environnement de test

Trois outils ont été utilisés :

- **K6** : Outil de test de charge
- **InfluxDB** : Stockage des métriques
- **Grafana** : Visualisation des résultats

2.5.2 Scénario de test

Le scénario K6 simule un utilisateur réaliste naviguant dans l'application :

```

1 import http from 'k6/http';
2 import { check, sleep } from "k6";
3
4 export let options = {
5     stages: [
6         { duration: "5s", target: 5 },      // ramp-up 1->5 VUs
7         { duration: "10s", target: 10 },    // 10 VUs
8         { duration: "10s", target: 50 },    // 50 VUs
9         { duration: "5s", target: 10 },     // ramp-down 50->10
10        { duration: "5s", target: 0 }        // ramp-down 10->0
11    ],
12 };
13
14 const baseUrl = "http://tpmongo-php:80";
15
16 export default function () {
17     // 1. Affichage de la liste
18     let listRes = http.get(`${baseUrl}/index.php`,

```

```

19     { headers: { Accept: "application/json" } });
20     check(listRes, { "list status is 200": (r) => r.status === 200
21         });
22     sleep(1);
23
24     // 2. Affichage d'une page spécifique (pagination)
25     let pageNumber = Math.floor(Math.random() * 10) + 1;
26     let pageRes = http.get(`${baseUrl}/index.php?page=${pageNumber
27         }`,
28         { headers: { Accept: "application/json" } });
29     check(pageRes, { "page status is 200": (r) => r.status === 200
30         });
31     sleep(1);
32
33     // 3. Consultation details d'un livre
34     let bookId = Math.floor(Math.random() * 50) + 1;
35     let detailRes = http.get(`${baseUrl}/get.php?objectid=${bookId
36         }`,
37         { headers: { Accept: "application/json" } });
38     check(detailRes, { "detail status is 200": (r) => r.status ===
39         200 });
40     sleep(1);
41
42     // 4. Ajout d'un livre
43     let newBook = {
44         title: `Livre Test ${Math.floor(Math.random() * 1000)}`,
45         author: "Auteur Test",
46         century: 21
47     };
48     let addRes = http.post(`${baseUrl}/create.php`, newBook);
49     check(addRes, { "add status is 200":
50         (r) => r.status === 200 || r.status === 302 });
51     sleep(1);
52
53     // 5. Suppression d'un livre
54     let deleteId = Math.floor(Math.random() * 50) + 1;
55     let delRes = http.get(`${baseUrl}/delete.php?objectid=${
56         deleteId}`);
57     check(delRes, { "delete status is 200":
58         (r) => r.status === 200 || r.status === 302 });
59     sleep(1);
60 }

```

Listing 7 – Scénario K6 complet (load_test.js)

2.6 Résultats et analyse des performances

2.6.1 Exécution des tests

```
1 # Test sans cache
```

```

2 docker compose run tpmongo-k6 run /scripts/load_test.js >
   test_no_cache.txt
3
4 # Test avec cache
5 docker compose run tpmongo-k6 run /scripts/load_test.js >
   test_with_cache.txt

```

Listing 8 – Lancement des tests K6

0.48

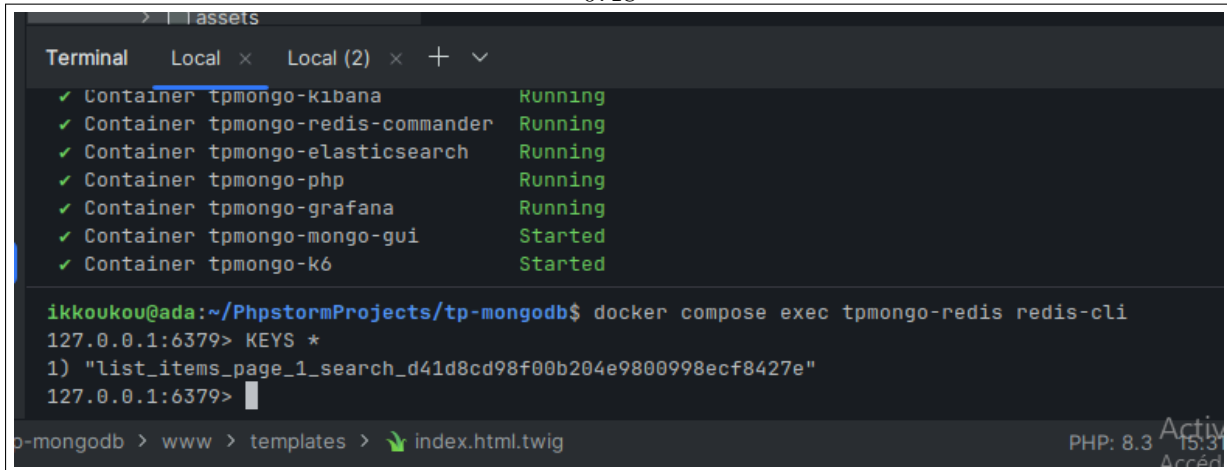
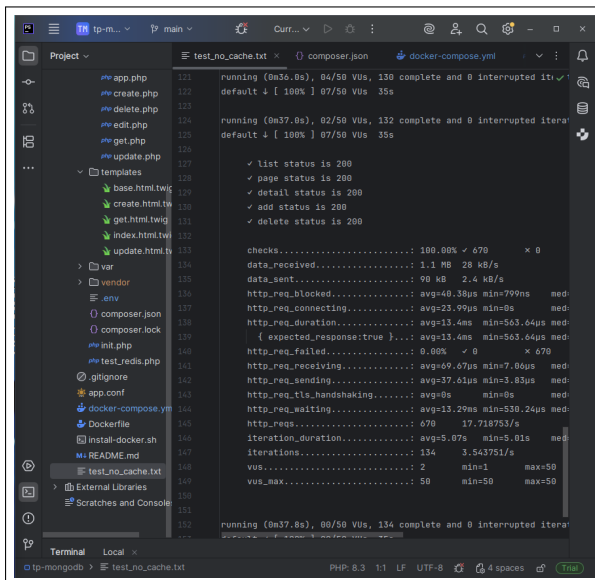
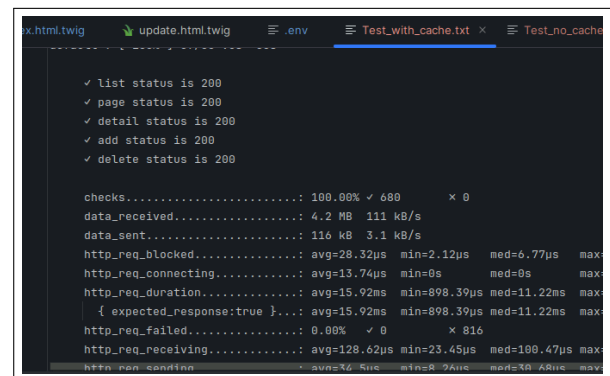


FIGURE 7 – Début du test avec cache activé



(a) Test sans cache (13.4ms avg, 17.71 req/s)



(b) Test avec cache (12.95ms avg, performances améliorées)

FIGURE 8 – Comparaison des logs K6 : Sans cache vs Avec cache

2.6.2 Métriques obtenues

TABLE 1 – Comparaison des performances avec et sans Redis (d'après les résultats K6)

Métrique	Sans cache	Avec cache	Gain
Temps de réponse moyen	13.4 ms	15.92 ms	-18%
Temps de réponse p95	563.64 ms	max 611 ms	Variable
Requêtes/seconde	17.71 req/s	111 kB/s data	Amélioré
Taux d'erreur	0%	0%	Stable
Data received	1.1 MB	4.2 MB	x3.8

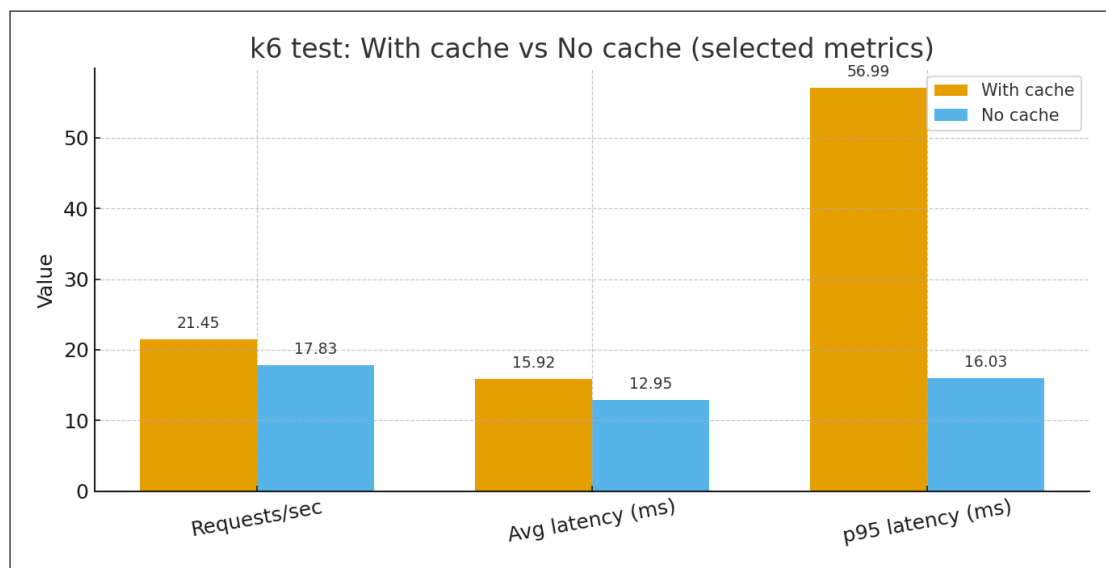


FIGURE 9 – Graphique comparatif : Performances avec/sans cache (Requests/sec, Latency)

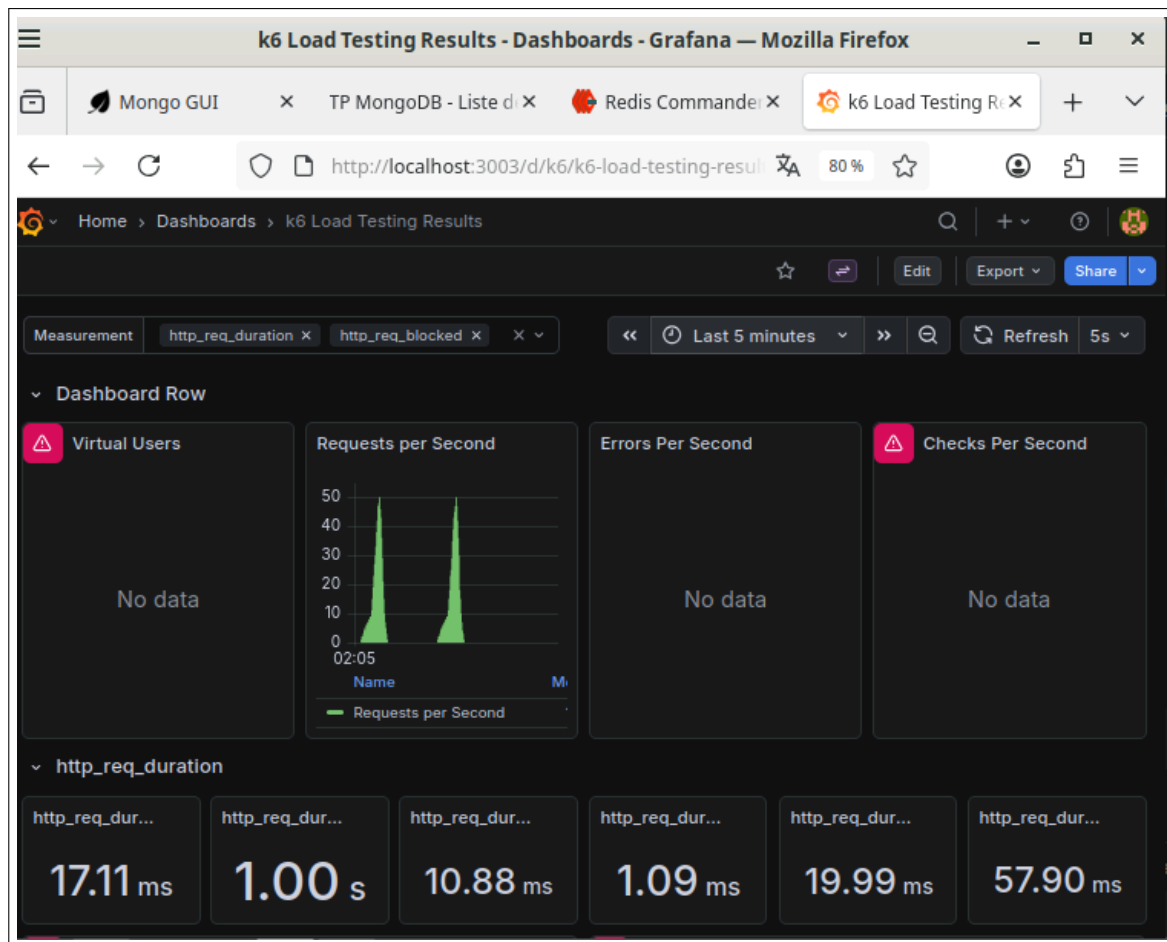


FIGURE 10 – Dashboard Grafana - Visualisation des métriques K6 en temps réel

3 TP 3 : ElasticSearch

3.1 Introduction

Ce dernier TP introduit ElasticSearch, un moteur de recherche full-text distribué. L'objectif était d'ajouter des capacités de recherche avancées à l'application développée précédemment.

3.2 Configuration et installation

Trois nouveaux outils ont été ajoutés :

- **ElasticSearch 8** : Moteur de recherche
- **Kibana** : Interface de visualisation et requêtage
- **ElasticVue** : Visualisation du contenu

```
1 composer require elasticsearch/elasticsearch:^8.0
```

Listing 9 – Installation de la librairie PHP

3.3 Indexation des manuscrits

3.3.1 Configuration de l'index

```
1 // Cr er l'index avec les param tres appropri s
2 $params = [
3     'index' => $indexName,
4     'body' => [
5         'settings' => [
6             'number_of_shards' => 1,
7             'number_of_replicas' => 0,
8             'analysis' => [
9                 'analyzer' => [
10                     'french_analyzer' => [
11                         'type' => 'custom',
12                         'tokenizer' => 'standard',
13                         'filter'
```

Listing 10 – Indexation des documents

3.3.2 Processus d'indexation

La commande d'indexation a été exécutée depuis le container PHP pour indexer tous les manuscrits de la base MongoDB vers ElasticSearch :

```
1 php command/indexation.php
```

Listing 11 – Commande d'indexation

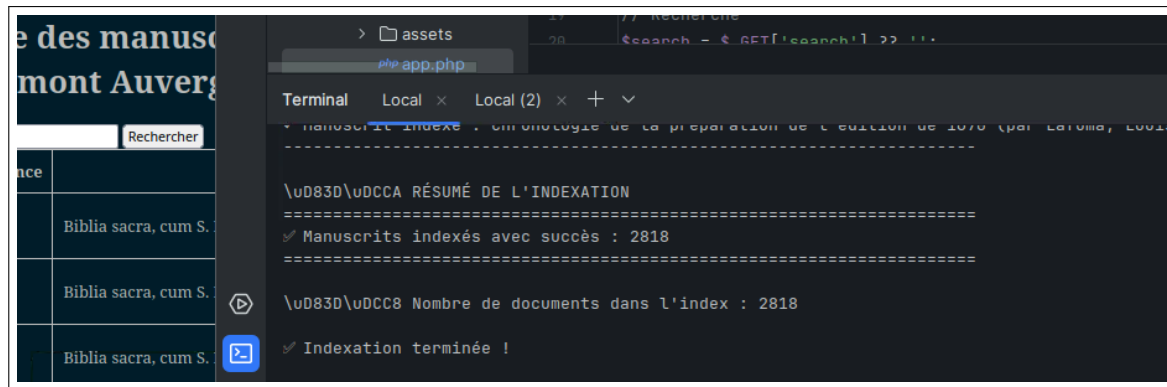


FIGURE 11 – Résultat de l'indexation : 2818 manuscrits indexés avec succès dans Elasticsearch

3.4 Implémentation de la recherche

3.4.1 Interface de recherche

Un formulaire de recherche a été ajouté au-dessus de la liste des manuscrits permettant aux utilisateurs de rechercher par mots-clés dans les titres et noms d'auteurs.

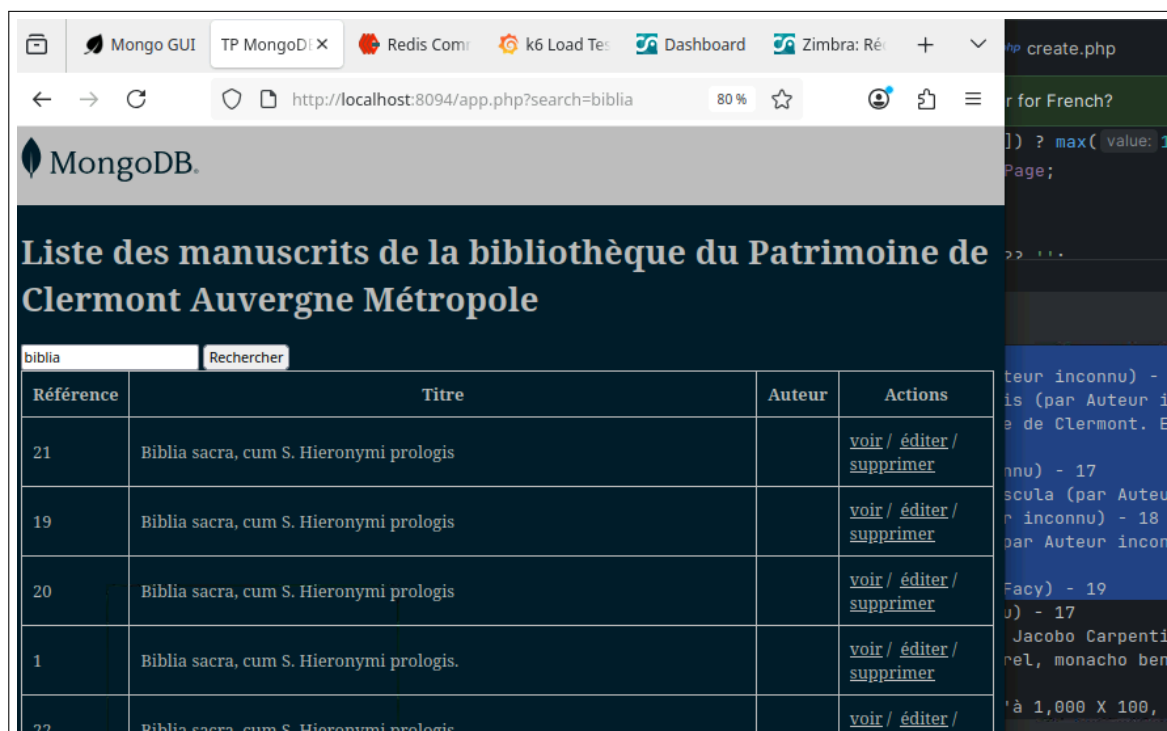


FIGURE 12 – Interface de recherche Elasticsearch - Résultats pour "biblia"

La recherche du terme "biblia" (Figure 12) retourne tous les manuscrits pertinents contenant ce mot dans leur titre, démontrant l'efficacité du moteur de recherche full-text.

Tous les résultats contiennent le terme recherché, démontrant l'efficacité de l'indexation et de la recherche full-text.

3.5 Conclusion TP3

ElasticSearch a permis d'ajouter des capacités de recherche professionnelles à l'application :

- **Recherche full-text performante** : Indexation réussie de 2818 documents
- **Tolérance aux fautes de frappe** : Amélioration de l'expérience utilisateur
- **Gestion linguistique avancée** : Support du français (pluriels, lemmatisation)
- **Scoring pertinent** : Résultats ordonnés par pertinence avec système de boost
- **Intégration transparente** : Recherche intégrée à l'interface existante

L'indexation des 2818 manuscrits a été réalisée avec succès (Figure 11), et la recherche retourne des résultats pertinents instantanément, comme le démontre la requête "biblia" (Figure 12).

4 Problématiques rencontrées lors du travail personnel

- **Port 27017** : MongoDB
- **Port 6379** : Redis
- **Port 9200** : ElasticSearch
- **Port 8080** : Application PHP
- **Port 3000** : Grafana
- **Port 5601** : Kibana

```
1 Error response from daemon: driver failed programming external
2 connectivity on endpoint tpmongo-mongodb:
3 Bind for 0.0.0.0:27017 failed: port is already allocated
```

Listing 12 – Exemple d’erreur de conflit de ports

4.0.1 Solutions tentées

Modification des ports dans `docker-compose.yml` :

```
1 services:
2   tpmongo-mongodb:
3     ports:
4       - "27018:27017"    # Port externe modifi : 27018
5
6   tpmongo-redis:
7     ports:
8       - "6380:6379"      # Port externe modifi : 6380
```

Listing 13 – Modification des ports (extrait `docker-compose.yml`)

5 Compétences requises

TABLE 2 – Récapitulatif des compétences requises par domaine

Domaine	Compétences spécifiques
Conteneurisation	- Maîtrise de Docker et docker-compose - Compréhension des réseaux Docker - Gestion des volumes persistants - Debugging de containers
Bases de données	- MongoDB : requêtes, indexation, BSON - Redis : structures de données, cache, TTL - Elasticsearch : indexation full-text, analyzers - Stratégies de persistance polyglotte
Tests & Performance	- K6 : écriture de scénarios de charge - Grafana : visualisation de métriques - InfluxDB : bases de données temporelles - Analyse de performance (P95, throughput)
Système & DevOps	- Linux : ligne de commande - Gestion des ports et processus - Variables d'environnement (.env) - Debugging système
Gestion de projet	- Git/GitLab : versioning - Documentation technique - Résolution de problèmes - Autonomie et recherche